

Estructuras de datos en JavaScript:

Aplicaciones integradas con estructuras de datos en JavaScript utilizando métodos avanzados y desestructuración de arreglos y objetos.

Situación - El Dashboard de E-Commerce

Imagina que estás construyendo el panel de control de Amazon. Tienes una lista de 50 productos que acaba de llegar desde el servidor (API). El gerente de ventas te pide tres cosas:

1. Mostrar en pantalla solo los productos que tienen descuento.
2. Crear tarjetas visuales (HTML) para cada uno de esos productos.
3. Agregar un nuevo producto al catálogo porque acaba de llegar un cargamento.

En la programación tradicional (la que vimos la sesión pasada), usaríamos un ciclo for para buscar los descuentos, concatenaríamos texto manualmente, y usaríamos push() para meter el producto nuevo.

Pero en la industria moderna y en frameworks como **React** (que usarán el próximo semestre), existe una regla de oro llamada **Inmutabilidad**:

"Nunca modifiques los datos originales, crea copias exactas con los cambios aplicados".

Si usas push() en React, la pantalla simplemente no se actualizará. Hoy aprenderemos a manipular datos como profesionales usando el **Spread Operator** para hacer copias, y los métodos **Map** y **Filter** para transformar listas en milisegundos.

La Inmutabilidad y el Spread Operator (...)

El operador de propagación (Spread) son literalmente tres puntos Su trabajo es "desempacar" o "esparcir" los elementos de un arreglo u objeto dentro de uno nuevo. Nos permite hacer copias reales sin modificar el original.

```
const carritoAntiguo = ["Laptop", "Mouse"];
```

```
// Forma antigua (Muta el arreglo original, prohibido en React)  
// carritoAntiguo.push("Teclado");
```

```
// Forma moderna (Inmutabilidad: Crea un arreglo nuevo)  
const carritoActualizado = [...carritoAntiguo, "Teclado"];
```

```
console.log(carritoAntiguo); // ["Laptop", "Mouse"] (Quedó intacto)  
console.log(carritoActualizado); // ["Laptop", "Mouse", "Teclado"]
```

Desestructuración en Parámetros

La desestructuración es una sintaxis de asignación. No muta el arreglo u objeto original; simplemente crea nuevas variables apuntando a los valores internos. Se basa en las **claves (nombres)** de las propiedades. El orden no importa.

Las bases de datos suelen enviar nombres de variables en formatos como `user_first_name`. Podemos renombrarlos sobre la marcha usando `:`.

```
const servidor = {  
  host: "localhost",  
  puerto: 8080,  
  db: { user: "admin", pass: "1234" }  
};  
  
// Extracción, renombramiento (host a url) y valor por defecto (ssl = true)  
const { host: url, puerto, ssl = true } = servidor;  
  
// Desestructuración anidada (extrayendo user directo desde db)  
const { db: { user } } = servidor;  
  
console.log(url); // "localhost"  
console.log(ssl); // true  
console.log(user); // "admin"
```

Desestructuración de Arreglos

Se basa estrictamente en la **posición (índice)** de los elementos. El nombre de la variable lo define el desarrollador.

Extracción posicional: El primer nombre de variable tomará el índice 0, el segundo el índice 1, etc.

Omitir elementos: Se pueden saltar posiciones dejando espacios vacíos entre comas , ,.

Operador Rest (...): Agrupa el "resto" de los elementos no extraídos en un nuevo arreglo.

```
const configuracion = ["192.168.1.1", "255.255.255.0", "GoogleDNS", "CloudflareDNS"];
```

```
// Extracción básica y salto del segundo elemento  
const [ip, , dnsPrimario] = configuracion;
```

```
// Operador Rest  
const [direccion, ...restoConfig] = configuracion;
```

```
console.log(ip); // "192.168.1.1"  
console.log(restoConfig); // ["255.255.255.0", "GoogleDNS", "CloudflareDNS"]
```

El Método .map()

.map() toma un arreglo, le aplica una función a **cada uno** de sus elementos, y devuelve un **arreglo nuevo** exactamente del mismo tamaño, pero transformado.

- **Caso de uso:** Convertir una lista de datos (JSON) en una lista de etiquetas HTML para mostrar en pantalla.

Anatomía de .map():

```
const precios = [100, 200, 300];
```

```
// Recorremos los precios y devolvemos cada uno multiplicado por 1.19 (IVA)
```

```
const preciosConIva = precios.map(precio => precio * 1.19);
```

```
console.log(preciosConIva); // [119, 238, 357]
```

El Método .filter()

.filter() toma un arreglo y devuelve un arreglo nuevo (generalmente más pequeño) que contiene solo los elementos que pasaron una prueba lógica (true).

- Caso de uso: Buscadores, filtros de "precio menor a", o mostrar solo usuarios activos.

Anatomía de .filter():

```
const edades = [15, 22, 17, 30];
```

```
// Guarda solo las edades que sean mayores o iguales a 18
```

```
const mayoresDeEdad = edades.filter(edad => edad >= 18);
```

```
console.log(mayoresDeEdad); // [22, 30]
```

Preguntas Frecuentes (FAQs)

¿Cuál es la diferencia entre `.forEach()` y `.map()`?

`.forEach()` es como un trabajador que hace una tarea y se va (ej. imprimir en consola); **no devuelve nada** (`undefined`).

`.map()` es como una fábrica: entra materia prima y **devuelve un producto nuevo** (un nuevo arreglo). Si necesitas guardar el resultado en una variable, DEBES usar `.map()`.

¿Por qué en la desestructuración de arreglos uso corchetes `[]` y en los objetos uso llaves `{}` al lado izquierdo del igual?

El símbolo de la izquierda debe imitar la estructura de la derecha. Si desempacas un arreglo `[1, 2]`, usas `[a, b]`. Si desempacas un objeto `{x: 1}`, usas `{x}`.

Ejemplos Demostrativos

Vamos a simular un flujo completo de consumo de API, combinando todos los conceptos.

Paso 1: Tenemos los datos de la API

```
const baseDeDatos = [  
  { id: 1, titulo: "JavaScript Ninja", precio: 40, stock: true },  
  { id: 2, titulo: "React Avanzado", precio: 60, stock: false },  
  { id: 3, titulo: "CSS Master", precio: 30, stock: true }  
];
```

Paso 2: Filtrar la información (Dejar solo lo que hay en stock)

```
const librosDisponibles = baseDeDatos.filter(libro => libro.stock === true);  
// (También se puede escribir simplemente: libro => libro.stock)
```

Paso 3: Transformar los datos a HTML usando Map y Desestructuración en Parámetros.

// Observa cómo deseestructuramos {titulo, precio} directamente en el parámetro

```
const catalogoHTML = librosDisponibles.map(({ titulo, precio }) => {  
  return `  
    <article class="card">  
      <h3>${titulo}</h3>  
      <p>Precio: ${precio}</p>  
      <button>Comprar</button>  
    </article>  
  `;  
});
```

```
console.log("Tarjetas listas para inyectar al DOM:", catalogoHTML);
```

Paso 4: Agregar un nuevo libro sin dañar el original (Spread Operator)

```
const nuevoLibro = { id: 4, titulo: "NodeJS Backend", precio: 50, stock: true };
```

```
const nuevaBaseDeDatos = [...baseDeDatos, nuevoLibro];  
console.log("Nueva DB (Inmutable):", nuevaBaseDeDatos);
```

Práctica Acompañada - Learning by Doing - Catálogo de Películas Netflix:

Objetivo: Combinar `.filter()` y `.map()` con desestructuración para procesar un catálogo.

Copia el siguiente arreglo en tu código:

```
const peliculasAPI = [  
  { id: 1, titulo: "Interstellar", genero: "Sci-Fi", rating: 8.6 },  
  { id: 2, titulo: "Son como niños", genero: "Comedia", rating: 5.9 },  
  { id: 3, titulo: "Inception", genero: "Sci-Fi", rating: 8.8 }  
];
```

- Usa `.filter()` para crear un nuevo arreglo llamado `peliculasTop` que contenga solo las películas con un rating mayor a 8.0.
- Toma ese nuevo arreglo (`peliculasTop`) y usa `.map()`. En los parámetros del map, **desestructura** el `{ titulo, genero }`.
- El `.map()` debe retornar un arreglo de strings con el formato: "Recomendada: [titulo] - Categoría: [genero]". Guarda esto en una variable `recomendaciones`.
- Imprime recomendaciones en la consola.

Código de Solución:

```
const peliculasAPI = [  
  { id: 1, titulo: "Interstellar", genero: "Sci-Fi", rating: 8.6 },  
  { id: 2, titulo: "Son como niños", genero: "Comedia", rating: 5.9 },  
  { id: 3, titulo: "Inception", genero: "Sci-Fi", rating: 8.8 }  
];  
  
// 1. Filtrar  
const peliculasTop = peliculasAPI.filter(pelicula => pelicula.rating > 8.0);  
  
// 2. Mapear y Desestructurar  
const recomendaciones = peliculasTop.map(({ titulo, genero }) => {  
  return `Recomendada: ${titulo} - Categoría: ${genero}`;  
});  
  
console.log(recomendaciones);  
// ["Recomendada: Interstellar - Categoría: Sci-Fi", "Recomendada: Inception - Categoría: Sci-Fi"]
```

Actividad de clase

Pair Programming:

- **Driver:** Administra la sintaxis de desestructuración en el editor.
- **Navigator:** Valida los nombres de las claves y la correcta anidación.

El Reto:

1. Copien este objeto base:
`const reporteClima = { ciudad: "Medellín", temp: { actual: 24, max: 28, min: 18 },
humedad: 60 };`
2. Utilicen una sola línea de desestructuración para extraer: ciudad, actual (renombrada a temperaturaActual) y una propiedad ficticia viento (asignándole un valor por defecto de "10 km/h").
3. Impriman las tres variables en un `console.log` usando backticks (Template strings).

Práctica Integradora Supervisada

Ticket JIRA-123: Limpieza de Props de Componente

Tienes una función flecha Header que recibe un objeto inmenso. Modifica los parámetros de la función para desestructurar solo título y subtítulo. Si subtítulo no viene, asigna "Bienvenido" por defecto.

Código Base:

```
const props = { titulo: "Dashboard", url: "/home", activo: true };

// Modifica los parámetros aquí
const Header = (propiedades) => {
  return `

# ${propiedades.titulo}</h1><h2>${propiedades.subtitulo}</h2>`; };


```

Solución Completa:

```
const props = { titulo: "Dashboard", url: "/home", activo: true };

const Header = ({ titulo, subtítulo = "Bienvenido" }) => {
  return `

# ${titulo}</h1><h2>${subtítulo}</h2>`; }; console.log(Header(props));


```

Troubleshooting (Solución de Errores Comunes)

El map que devuelve un arreglo de undefined:

- Error: El estudiante hace el map con llaves {} pero olvida la palabra return. `datos.map(item => { item.precio * 2; })`
- Solución: Si abres llaves en un Arrow Function, el return es obligatorio. Si es de una sola línea, quita las llaves: `datos.map(item => item.precio * 2)`.

Desestructurar desde el vacío (Destructuring of undefined):

- Error: Intentar mapear y desestructurar una propiedad que no existe, o si el arreglo está vacío.
- Consecuencia: Arrojará `TypeError: Cannot destructure property`. Se debe asegurar que la estructura de datos que llega de la API es la esperada antes de desestructurarla ciegamente.

Spread Operator con llaves equivocadas:

- Error: Intentar usar spread de un objeto dentro de un arreglo directamente sin envolverlo en un objeto nuevo.
- Solución: Para arreglos se usa `[...arr]`. Para objetos se usa `{...obj}`. No los cruces incorrectamente.

Ejercicio Independiente Aplicado

Misión: "Panel de Control (To-Do List) Profesional"

Instrucciones: Estás creando la lógica de una app de tareas tipo Trello. Tienes la siguiente base de datos:

```
const listaTareas = [  
  { id: 1, texto: "Aprender HTML", completada: true },  
  { id: 2, texto: "Aprender CSS", completada: true },  
  { id: 3, texto: "Dominar React", completada: false }  
];
```

Paso 1: El usuario agrega una nueva tarea: "Estudiar APIs". Usando el **Spread Operator**, crea un arreglo llamado `tareasActualizadas` que incluya la nueva tarea (con `completada: false` y un `id: 4`).

Paso 2: El usuario hace clic en el filtro "Pendientes". Crea una variable `tareasPendientes` usando `.filter()` sobre las `tareasActualizadas` para guardar solo las que tengan `completada === false`.

Paso 3: Necesitamos renderizar esto en la pantalla. Usa `.map()` sobre las `tareasPendientes`, desestructura la propiedad `{ texto }` en los parámetros, y devuelve un arreglo de strings HTML: `"<li class='pendiente'>[texto]</li>"`.

Encadenamiento (Chaining)

El secreto del encadenamiento radica en lo que **devuelve (retorna)** cada método.

- El método `.filter()` devuelve un **Arreglo nuevo**.
- El método `.map()` pertenece a los arreglos y devuelve otro **Arreglo nuevo**.

Como `.filter()` te entrega un arreglo en la mano, puedes inmediatamente invocar `.map()` sobre ese resultado en la misma línea (o la línea siguiente), sin necesidad de darle un nombre a ese paso intermedio.

Para que el código encadenado sea legible (y no una línea kilométrica horizontal que nadie puede leer), la regla de la industria es dar un **salto de línea y tabular** antes de cada punto .

```
const resultadoFinal = arregloOriginal
    .metodoUno()
    .metodoDos()
    .metodoTres();
```

¿Puedo encadenar `.forEach()` al final o en el medio de un encadenamiento?

Puedes ponerlo al *final*, pero **nunca en el medio**. Recuerda que `.forEach()` retorna `undefined`. Si haces `.filter().forEach().map()`, el `.map()` intentará ejecutarse sobre `undefined` y tu programa se romperá lanzando un error fatal.

¿El encadenamiento hace que el código sea más rápido para la computadora?

No, de hecho, el rendimiento es exactamente el mismo que si usaras variables temporales, porque JavaScript sigue recorriendo el arreglo múltiples veces por debajo. La ventaja del encadenamiento es estrictamente **humana**: hace que el código sea más limpio, más fácil de leer y evita que llenes la memoria de nombres de variables inútiles que solo usas una vez.